JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class PipedOutputStream

java.lang.Object
    java.io.OutputStream
        java.io.PipedOutputStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `AutoCloseable`

```
public class PipedOutputStream
extends OutputStream
```

A piped output stream can be connected to a piped input stream to create a communications pipe. The piped output stream is the sending end of the pipe. Typically, data is written to a `PipedOutputStream` object by one thread and data is read from the connected `PipedInputStream` by some other thread. Attempting to use both objects from a single thread is not recommended as it may deadlock the thread. The pipe is said to be *broken* if a thread that was reading data bytes from the connected piped input stream is no longer alive.

Since:

`JDK1.0`

See Also:

`PipedInputStream`

Constructor Summary

Constructors

Constructor	Description
`PipedOutputStream()`	Creates a piped output stream that is not yet

	connected to a piped input stream.
PipedOutputStream(**PipedInputStream** snk)	Creates a piped output stream connected to the specified piped input stream.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	**close**()	Closes this piped output stream and releases any system resources associated with this stream.
void	**connect**(**PipedInputStream** snk)	Connects this piped output stream to a receiver.
void	**flush**()	Flushes this output stream and forces any buffered output bytes to be written out.
void	**write**(byte[] b, int off, int len)	Writes `len` bytes from the specified byte array starting at offset `off` to this piped output stream.
void	**write**(int b)	Writes the specified `byte` to the piped output stream.

Methods inherited from class java.io.OutputStream

write

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

PipedOutputStream

```
public PipedOutputStream(PipedInputStream snk)
                  throws IOException
```

Creates a piped output stream connected to the specified piped input stream. Data bytes written to this stream will then be available as input from `snk`.

Parameters:

`snk - The piped input stream to connect to.`

Throws:

`IOException - if an I/O error occurs.`

PipedOutputStream

```
public PipedOutputStream()
```

Creates a piped output stream that is not yet connected to a piped input stream. It must be connected to a piped input stream, either by the receiver or the sender, before being used.

See Also:

PipedInputStream.connect(java.io.PipedOutputStream), connect(java.io.PipedInputStream)

Method Detail

connect

```
public void connect(PipedInputStream snk)
             throws IOException
```

Connects this piped output stream to a receiver. If this object is already connected to some other piped input stream, an `IOException` is thrown.

If `snk` is an unconnected piped input stream and `src` is an unconnected piped output stream, they may be connected by either the call:

```
src.connect(snk)
```

or the call:

```
snk.connect(src)
```

The two calls have the same effect.

Parameters:

`snk - the piped input stream to connect to.`

Throws:

`IOException - if an I/O error occurs.`

write

```
public void write(int b)
           throws IOException
```

Writes the specified `byte` to the piped output stream.

Implements the `write` method of `OutputStream`.

Specified by:

`write in class OutputStream`

Parameters:

`b - the byte to be written.`

Throws:

`IOException - if the pipe is broken, unconnected, closed, or if an I/O error occurs.`

write

```
public void write(byte[] b,
                  int off,
                  int len)
           throws IOException
```

Writes `len` bytes from the specified byte array starting at offset `off` to this piped output stream. This method blocks until all the bytes are written to the output stream.

Overrides:

`write in class OutputStream`

Parameters:

b - the data.

off - the start offset in the data.

len - the number of bytes to write.

Throws:

IOException - if the pipe is broken, unconnected, closed, or if an I/O error occurs.

flush

```
public void flush()
           throws IOException
```

Flushes this output stream and forces any buffered output bytes to be written out. This will notify any readers that bytes are waiting in the pipe.

Specified by:

flush in interface Flushable

Overrides:

flush in class OutputStream

Throws:

IOException - if an I/O error occurs.

close

```
public void close()
           throws IOException
```

Closes this piped output stream and releases any system resources associated with this stream. This stream may no longer be used for writing bytes.

Specified by:

close in interface Closeable

Specified by:

close in interface AutoCloseable

Overrides:

close in class OutputStream

Throws:

IOException - if an I/O error occurs.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.